

TapHome +



Home Assistant

Local Integration Guide

Step-by-step installation, API discovery, YAML configuration, validation and troubleshooting

Prepared by Jomar Mireles

[LinkedIn: linkedin.com/in/jomarjmm](https://www.linkedin.com/in/jomarjmm)

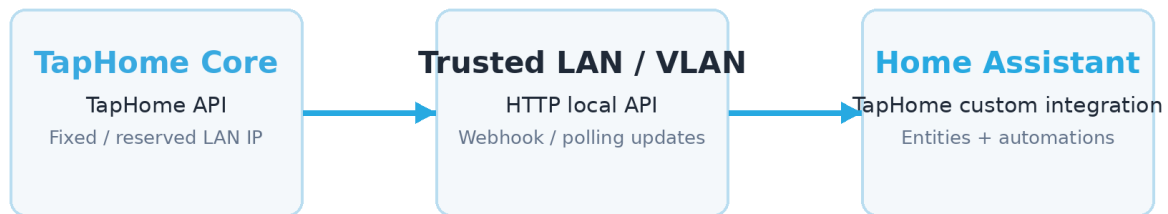
Scope This guide is installation-neutral. It contains no site-specific names, addresses, IP addresses, device inventories or screenshots from a private installation. Replace every placeholder with values from the target installation.

Version 1.0 - September 2026

1. Purpose and expected result

This guide describes how to expose TapHome devices through the TapHome API and integrate them into Home Assistant using the community TapHome custom integration. The recommended design is local: Home Assistant communicates with the TapHome Core over the trusted LAN, while a webhook can provide immediate state updates.

Recommended local architecture



Preferred: local communication over a trusted network. Avoid exposing the Core API directly to the Internet.

What you should have at the end

- A working TapHome API token and a known Core LAN address.
- A machine-readable inventory of exposed devices and their Device IDs.
- The TapHome custom integration installed in Home Assistant.
- Valid YAML mapping TapHome device IDs to Home Assistant platforms.
- Working lights, covers, switches, sensors and binary sensors with clean startup logs.

Important The Home Assistant TapHome component used in this guide is a community/custom integration, not a Home Assistant Core integration. Keep backups and review its changelog before upgrades.

2. Prerequisites and security

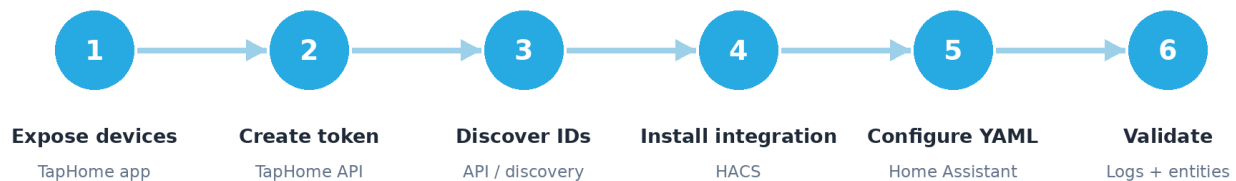
2.1 Required components

- TapHome Core running a current firmware release.
- TapHome app access with permission to configure Expose Devices.
- Home Assistant with file access to configuration.yaml and secrets.yaml.
- HACS installed (recommended) or the ability to install a custom component manually.
- LAN connectivity between Home Assistant and the TapHome Core.
- A DHCP reservation or static address for the Core is strongly recommended.

2.2 Security rules

- Never publish the TapHome API token in screenshots, forum posts, GitHub commits or documentation.
- Store the token in Home Assistant secrets.yaml and reference it with !secret.
- The local Core API is HTTP. Use it only on a trusted LAN/VLAN or over a trusted VPN; do not port-forward it to the public Internet.
- Expose only the devices that the external system actually needs.
- If a token is compromised, regenerate it in TapHome. A newly generated token invalidates the previous one.

Installation workflow



Key rule

Expose only what you need, keep the token secret, and validate each HA platform independently.

3. Enable and use the TapHome API

3.1 Expose devices in TapHome

1. Open the TapHome application for the target installation.
2. Go to Settings > Expose devices.
3. Add or open the TapHome API interface.
4. Use Add device to select each device that should be available to Home Assistant.
5. Generate or copy the API token. Store it immediately in a password manager or other secure location.
6. Record the Core LAN IP address. Prefer a DHCP reservation so it does not change.

Token warning Do not include a real token in this guide, a Git repository, issue report or chat transcript. Use placeholders such as <TAPHOME_TOKEN>.

3.2 API base URLs

TapHome documents the cloud API at <https://api.taphome.com/api/TapHomeApi/v1/>. For local access, the community Home Assistant integration used in this guide documents the Core URL in the form below:

```
http://<CORE_IP>/api/cloudapi/v1
```

TapHome firmware/API documentation also refers to `/api/TapHomeApi/v1` and `/api/CloudApi/v1` endpoint paths. For Home Assistant, follow the custom integration configuration documentation and test the chosen base URL with `/location` or `/discovery`. Do not assume that cloud and local URLs are interchangeable on every firmware release.

3.3 Test authentication

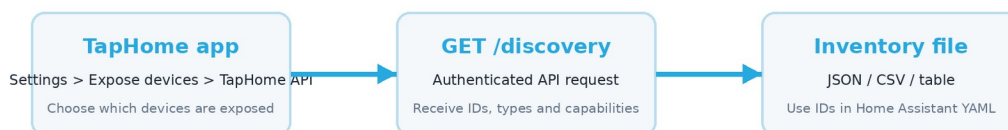
```
curl -s \
  -H "Authorization: TapHome <TAPHOME_TOKEN>" \
  "http://<CORE_IP>/api/cloudapi/v1/location" | python3 -m json.tool
```

A successful response confirms that the Core is reachable and the token is accepted.

4. Discover the installed/exposed equipment

The discovery endpoint is the safest way to obtain the exact devices that are available through the API. It returns each exposed device, its numeric deviceId, type/name/description information, and the supported value types. Those IDs are what you map in Home Assistant.

How to obtain the installed-device inventory



Discovery gives you the exact mapping you need

deviceId -> TapHome device identity used in YAML
supportedValues -> valid ValueType IDs and names
name / description / usage -> human-friendly identification

4.1 Get the complete discovery JSON

```
curl -s \
-H "Authorization: TapHome <TAPHOME_TOKEN>" \
"http://<CORE_IP>/api/cloudapi/v1/discovery" \
| python3 -m json.tool
```

4.2 Save the inventory to a file

```
curl -s \
-H "Authorization: TapHome <TAPHOME_TOKEN>" \
"http://<CORE_IP>/api/cloudapi/v1/discovery" \
> taphome-discovery.json
```

4.3 Produce a compact CSV inventory

```
python3 - <<'PY'
import json, csv
with open('taphome-discovery.json', encoding='utf-8') as f:
    data = json.load(f)
with open('taphome-inventory.csv', 'w', newline='', encoding='utf-8') as f:
    w=csv.writer(f)
    w.writerow(['deviceId', 'type', 'name', 'description', 'supportedValueTypes'])
    for d in data.get('devices', []):
        vals='; '.join(f"{v.get('valueTypeId')}: {v.get('valueTypeName')}" for v in
d.get('supportedValues', []))
        w.writerow([d.get('deviceId'), d.get('type'), d.get('name'), d.get('description'), vals])
print('Created taphome-inventory.csv')
PY
```

Why discovery matters Do not guess IDs or ValueTypes. Discovery is the authoritative API view of what the Core is exposing at that moment.

5. Read device values directly from the API

TapHome API: what each endpoint is for

location	Connectivity / location metadata Use first to verify authentication and reachability.
discovery	Inventory of exposed devices Returns deviceId, type, name and supported value types.
getDeviceValue/{id}	Current values for one device Best for troubleshooting a single entity.
getAllDevicesValues	Current values for all exposed devices Useful for inventory validation and diagnostics.
setDeviceValue	Write a supported value Use only after confirming the correct valueTypeId and semantics.

5.1 Read one device

```
curl -s \
  -H "Authorization: TapHome <TAPHOME_TOKEN>" \
  "http://<CORE_IP>/api/cloudapi/v1/getDeviceValue/<DEVICE_ID>" \
  | python3 -m json.tool
```

The response contains valueTypeId, valueTypeName and the current numeric value. This is the best diagnostic when a Home Assistant entity exists but is unavailable or does not update.

5.2 Read all exposed devices

```
curl -s \
  -H "Authorization: TapHome <TAPHOME_TOKEN>" \
  "http://<CORE_IP>/api/cloudapi/v1/getAllDevicesValues" \
  | python3 -m json.tool
```

5.3 Understand ValueType

A TapHome device can expose multiple values. For example, a contact sensor can expose DeviceStatus, OperationMode, ReedContact and BatteryPercentageRemaining. Home Assistant platforms must use only value types supported by both the device and the custom integration.

Critical A YAML entry containing an unsupported/invalid ValueType can prevent an entire Home Assistant TapHome platform (for example sensor or binary_sensor) from loading. If many sensors fail while lights/covers work, check the startup log for ValueError before changing integration code.

6. Install the TapHome custom integration

6.1 HACS method (recommended)

1. Open HACS in Home Assistant.
2. Open Integrations and search for TapHome.
3. Install the TapHome integration maintained in the martindybal/taphome-homeassistant repository.
4. Restart Home Assistant after installation.
5. Confirm that `custom_components/taphome` exists in the Home Assistant configuration directory.

6.2 Manual method

If HACS is not available, follow the repository instructions: create `custom_components/taphome` next to `configuration.yaml` and copy the integration files into that directory. Restart Home Assistant afterward.

Recommendation For third-party deployments, HACS is easier to maintain because updates are visible and can be reviewed before installation.

7. Configure Home Assistant securely

7.1 Store the API token in secrets.yaml

```
taphome_token: "PASTE_REAL_TOKEN_HERE"
```

7.2 Generic configuration.yaml example

```
taphome:
  cores:
    - id: site1
      token: !secret taphome_token
      api_url: http://<CORE_IP>/api/cloudapi/v1
      webhook_id: taphome_site1

      use_description_as_name: true
      use_description_as_entity_id: true

      enabled_attributes:
        - taphome_id
        - taphome_name
        - taphome_description
        - taphome_category
        - taphome_zone

      sensors:
        - <TEMPERATURE_DEVICE_ID>
        - <ENERGY_DEVICE_ID>

      binary_sensors:
        - id: <DOOR_CONTACT_ID>
          device_class: door
        - id: <MOTION_SENSOR_ID>
          device_class: motion

      lights:
        - <LIGHT_DEVICE_ID>

      covers:
        - id: <BLIND_DEVICE_ID>
          device_class: blind

      switches:
        - <SWITCH_DEVICE_ID>

      buttons:
        - <BUTTON_DEVICE_ID>
```

Replace every placeholder with a device ID found through /discovery. Start with a small set of known-good devices, validate, then expand the configuration.

8. Map TapHome devices to Home Assistant platforms

TapHome capability	Typical HA platform	Example YAML	Validation
Temperature / humidity / CO2 / energy	sensor	- 123	Value changes and unit looks correct
Door / window contact	binary_sensor	id: 123 device_class: door	Open/Closed changes physically
Motion	binary_sensor	id: 124 device_class: motion	Motion state clears/sets correctly
Light / dimmer	light	- 125	On/off and brightness
Blind / shutter	cover	id: 126 device_class: blind	Open/close/position
Relay / socket	switch	- 127	On/off command and feedback
Scene / action	button	- 128	Press performs expected action

8.1 Add devices incrementally

A practical commissioning method is to add one entity from each platform, validate Home Assistant, then add the remaining devices in groups. This makes it obvious which entry introduces a platform error.

8.2 Avoid unsupported ValueTypes

Use explicit `value_type` only when the integration documentation or the installed `ValueType` enum confirms the accepted value. Do not copy a `valueTypeName` string from the API and assume that the integration accepts that string in YAML.

9. Validate and commission the integration

9.1 Home Assistant Container

```
docker exec homeassistant python -m homeassistant --script check_config --config /config
docker restart homeassistant
docker logs homeassistant --since 10m 2>&1 | grep -i -E "taphome|ValueError|Error while setting up|
already used"
```

9.2 Home Assistant OS / Supervised

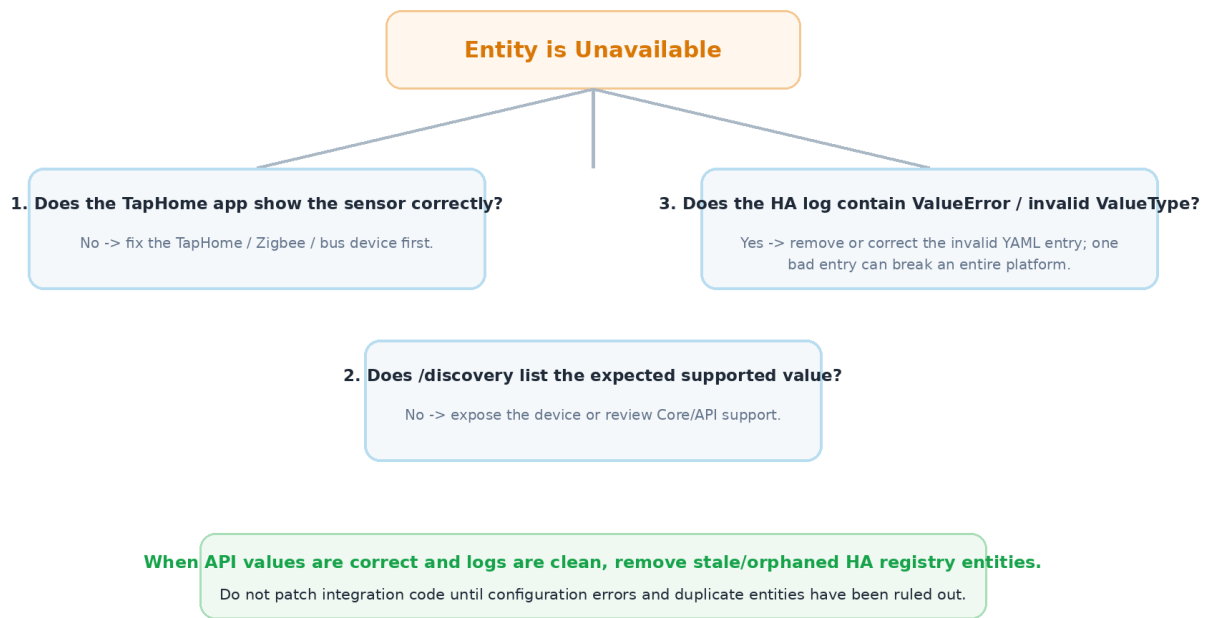
Use Settings > System > YAML configuration validation, then restart Home Assistant. Review Settings > System > Logs and filter for TapHome.

9.3 Functional test matrix

- Switch one light from Home Assistant and verify TapHome feedback.
- Move one blind/cover and confirm position or end state.
- Open and close one contact sensor and confirm the entity changes immediately.
- Trigger one motion sensor.
- Compare one temperature/CO2/energy value against the TapHome app.
- Press one mapped button/scene if configured.
- Filter TapHome entities by state = Unavailable and investigate anything that remains.

10. Troubleshooting

Troubleshooting sensor entities



10.1 If lights and covers work but sensors do not

This pattern usually means the Core/API connection is healthy. Check whether the sensor or binary_sensor platform failed during startup because of one invalid entry. Search for “Error while setting up taphome platform”, “ValueError”, or “is not a valid ValueType”. Remove/correct the offending entry and restart.

10.2 Duplicate unique ID

If the log says an ID is already used, the same TapHome value is being instantiated twice. Remove redundant YAML (for example explicit value_type when automatic discovery already creates the same entity) and restart. Then remove stale registry entities only after confirming which entity is active.

10.3 Orphaned entities

After configuration experiments, the Home Assistant entity registry can retain an old unavailable entity. Compare entity IDs, keep the working entity and delete only the stale unavailable registry entry.

11. Advanced API verification

11.1 Confirm a contact sensor directly

```
curl -s -H "Authorization: TapHome <TAPHOME_TOKEN>" \
  "http://<CORE_IP>/api/cloudapi/v1/getDeviceValue/<DEVICE_ID>" \
  | python3 -m json.tool
```

Operate the physical sensor and run the command again. If the expected valueTypeId changes between 0 and 1, the TapHome API side is working. Continue troubleshooting in the Home Assistant integration/configuration rather than the field device.

11.2 Inspect supported ValueTypes in the installed integration

```
grep -R "class ValueType" -n /config/custom_components/taphome
grep -R "REED|MOTION|TEMPERATURE" -n /config/custom_components/taphome/taphome_sdk/value_type.py
```

Container paths vary. On a Docker bind mount, the same files may also exist on the host under your Home Assistant configuration directory.

Change control Do not edit the Python integration as a first troubleshooting step. Keep the installed package unmodified whenever possible. Fix YAML, unsupported ValueTypes, duplicate entries and stale entities first.

12. Third-party installation handover checklist

- ☐ Core has a stable LAN address (static or DHCP reservation).
- ☐ Only required devices are exposed via TapHome API.
- ☐ API token is stored securely and is not present in documentation/screenshots.
- ☐ /location and /discovery work from the Home Assistant host.
- ☐ A copy of discovery JSON or a generated CSV inventory is stored with project documentation.
- ☐ HACS/custom integration version is recorded.
- ☐ Home Assistant YAML validates without TapHome platform errors.
- ☐ No TapHome entities remain unexpectedly Unavailable.
- ☐ Physical tests completed for each platform used.
- ☐ No router port-forward exposes the local API to the Internet.
- ☐ Backup of Home Assistant configuration completed before handover.

12.1 Maintenance recommendation

Before updating Home Assistant or the custom TapHome integration, review the repository changelog/issues and keep a current Home Assistant backup. After upgrades, repeat the log check and a small physical test set.

13. Sources, trademarks and credits

Primary references

- TapHome - Home Assistant Integration Guide: <https://taphome.com/en/docs/expose-devices/home-assistant/>
- TapHome - REST API for 3rd Party Apps: <https://taphome.com/en/docs/expose-devices/taphome-api/>
- TapHome - Expose Devices: <https://taphome.com/en/docs/expose-devices/>
- Community integration - martindybal/taphome-homeassistant: <https://github.com/martindybal/taphome-homeassistant/>
- Community integration configuration documentation: <https://github.com/martindybal/taphome-homeassistant/blob/development/configuration.md>
- Home Assistant: <https://www.home-assistant.io/>
- HACS: <https://www.hacs.xyz/>

Credits

Guide prepared by Jomar Mireles

[LinkedIn - Jomar Mireles](#)

Technical method based on TapHome public documentation, the community Home Assistant integration, and practical integration validation. No private installation names, addresses, IP addresses, tokens, inventories or site-specific screenshots are included.

Trademark notice

TapHome and Home Assistant names and logos are trademarks or brand assets of their respective owners. They are used in this guide only for identification and interoperability documentation. This guide is independent and does not claim endorsement by TapHome, Nabu Casa/Home Assistant, HACS, or the community integration maintainer.